

Week 1 note: Component B (The Runs)

CSIT332 Semester Project

Group 4 | Gazal, Brij, Heena

What we ran

run_pilot.py takes a stratified random sample of n_{sub} rows from `synthetic_pilot.csv`, splits it into 5 stratified folds, trains each of the six classifiers on four folds and scores AUC on the fifth. We did this for $n_{\text{sub}} = 50, 100, 200$ and 500 with 100 replicates each, which gives the 12,000 rows in `results.csv` (`dataset_id` is `synthetic_pilot` and `protocol_id` is `cv5`).

How long it took

The whole run took **239 seconds (about 4 minutes)** on a single CPU core with Python 3.12.3 and scikit-learn 1.8.0. By sample size it was roughly 45 s for $n=50$, 47 s for $n=100$, 55 s for $n=200$ and 92 s for $n=500$, so larger samples cost more but nothing was slow. The timer only covers the loop, not loading the csv. We did not see any warnings or errors.

Checks we did

`check_results.py` confirms that `results.csv` has exactly 12,000 rows in the right columns, no missing values, AUC between 0 and 1 and 5 folds for every classifier in every replicate. It also confirms that the same seed gives the same sample, that every train and test fold has both classes (the smallest test fold, at $n=50$, still has 2 positives), and that rerunning four replicates from scratch reproduces the stored AUCs.

Order of the classifiers and spread between replicates

Mean AUC over all folds and replicates for each classifier at each sample size:

classifier	n=50	n=100	n=200	n=500	sd at n=50	sd at n=500
logreg	0.695	0.727	0.752	0.778	0.113	0.028
knn	0.629	0.658	0.669	0.691	0.113	0.040
dtree	0.562	0.594	0.588	0.629	0.092	0.037
svm	0.599	0.713	0.757	0.785	0.187	0.034
rf	0.658	0.705	0.727	0.758	0.135	0.034
gb	0.617	0.685	0.713	0.761	0.136	0.037

The order at $n=500$ is svm, logreg, gb, rf, knn, dtree. The depth-5 decision tree is last at every sample size and knn is fifth from $n=100$ upwards, which seemed sensible to us. Every other classifier improves at each step up in n ; dtree is the exception, dipping a little from $n=100$ (0.594) to $n=200$ (0.588), which we think is just noise. The spread between replicates is large when n is small. For logreg the replicate-level AUC has a standard deviation of 0.113 at $n=50$ (values between 0.44 and 0.95) and 0.028 at $n=500$. So one small study can land almost anywhere.

What surprised us

1. The small-sample winners are not the large-sample winners. For our own reference (this is not the official truth from Component A) we trained each classifier once on 5,000 rows and tested it on 10,000 other rows. There gb was best (0.838), then svm (0.831), rf (0.812), logreg (0.789), knn (0.739) and dtree (0.726). But at $n=500$, gb had the top mean AUC in only 6 of 100 replicates, while logreg and svm together had it in 93. Our guess is that the simpler models get going with less data, but we have not tested that.

2. The SVM at $n=50$ is erratic. It has the biggest spread (sd 0.187, and single replicates as low as 0.20) and its AUC is below 0.5 in 149 of the 500 folds. With `probability=True`, the SVM's `predict_proba` ranking can disagree with its `decision_function` ranking. On the $n=50$ folds they differed in 106 of 500 folds, and the mean AUC was 0.599 with `predict_proba` against 0.682 with `decision_function`. We kept `predict_proba` because the instructions fix `probability=True` and say AUC, but it is worth knowing.

3. Ties. In 8 replicates (3 at $n=50$, 4 at $n=100$, 1 at $n=200$) two classifiers had exactly the same top mean AUC. With 10 rows in a test fold at $n=50$, fold AUCs can only take a few values, so ties happen. Also some AUCs come out as values like 0.7812499999999999, so anyone comparing classifiers with `==` will get it wrong and should use a tolerance.

Ambiguities and what we assumed

1. Instructions 1 asked for a written plan and said no code, and Instructions 2 asks for the runner. We followed Instructions 2 and from the old deliverables we only kept this list.
2. How to derive the seed is not given. We used $n_sub * 10000 + replicate_id$, for both the sample and the fold shuffle. A different formula would give different rows, so another group's file will not match ours row for row.
3. Whether ids start at 0 or 1. We used `replicate_id` 1 to 100 and `fold_id` 1 to 5.
4. How to draw a stratified sample. We used `train_test_split` with `stratify`, so the positive count can be one off from exactly 20% (10 positives at $n=50$).
5. Whether folds are shuffled. We used `StratifiedKFold` with `shuffle=True`.
6. Which score to use for AUC. We used `predict_proba[:, 1]` for all six, see point 2 above.
7. `random_state=0` is fixed inside the classifiers, so their own randomness is the same in every replicate. Only the sample and the folds change. We assumed that is intended.
8. Row order and precision of results.csv are not specified. We sorted by `n_sub`, `replicate`, `classifier`, `fold` and did not round the AUC.
9. Run time: we timed the loop only, on one core. A different machine will give a different number.